
University of Stavanger

Applied Robot Technology
ELE610

# Control robot from Python

RS09 Documentation

Judith Zaragoza López
282308

Áron Assani
287361

2025. 04. 22.

# Table of Contents

# 1 RAPID Code

## a) Framework

As a starting point for this assignment, the pick and place code from RS03 was used. The existing procedures were already almost completely usable, with only minor changes in need. The final version is as follows:

```
MODULE E458GripperTest
    ! Module for testing gripper with Python communication
    ! Adapted for puck handling with QR codes

    ! wobj in the middle of the table
    TASK PERS wobjdata
       wobjTableN:=[FALSE,TRUE,"",[[150,-500,8],[0.707106781,0,0,-0.707106781]],[[0,0,0]

    ! Variables for Python communication
    VAR num numWPW := 0;         ! What Python Wants
    VAR num numWRD := 0;         ! What RAPID Does
    VAR num numPuckX := 0;       ! Puck X position (from Python)
    VAR num numPuckY := 0;       ! Puck Y position (from Python)
    VAR num anglePlace := 0;     ! Puck angle position (from Python)
    VAR num numPlaceX := 0;      ! Place X position (from Python)
    VAR num numPlaceY := 0;      ! Place Y position (from Python)
    VAR num heightPlace := 0;    ! Place Z position (from Python)
    VAR string strPuckID := ""; ! Puck ID from QR code
    VAR num sideOffset := -50;

    ! Tool definitions
    ! TASK PERS tooldata
       tGripper:=[TRUE,[[0,0,114.25],[1,0,0,0]],[0.1,[0,0,0.1],[1,0,0,0],0,0,0]];
    TASK PERS tooldata
       tCamera:=[TRUE,[[0,0,114.25],[1,0,0,0]],[0.1,[0,0,0.1],[1,0,0,0],0,0,0]];

    ! Positions and speeds
    CONST robtarget
       p0:=[[0,0,250],[0,1,0,0],[0,0,0,0],[9E9,9E9,9E9,9E9,9E9,9E9]]; !
       Home position
    VAR robtarget aux := p0;
    CONST speeddata vFast:=v1000;
    CONST speeddata vSlow:=v5;
    CONST num puckHeight:=-240;
    CONST num safeHeight:=250;
    CONST num cameraHeight:=300;

    ! Puck positions (default place positions)
    VAR robtarget
       pPlace1:=[[0,0,250],[0,1,0,0],[0,0,0,0],[9E9,9E9,9E9,9E9,9E9,9E9]];
    VAR robtarget
       pPlace2:=[[200,0,0],[0,1,0,0],[0,0,0,0],[9E9,9E9,9E9,9E9,9E9,9E9]];
    VAR robtarget
       pPlace3:=[[200,200,0],[0,1,0,0],[0,0,0,0],[9E9,9E9,9E9,9E9,9E9,9E9]];

    PROC main()
        CloseGripper(FALSE); ! Open gripper
        MoveJ Offs(p0,0,0,safeHeight), vFast, z20,
           tGripper\WObj:=wobjTableN;
```

```
    ! Start main loop or test
    MainLoop;
ENDPROC

PROC MainLoop()
    TPWrite "MainLoop starts - Waiting for Python commands";
    numWRD := 0;
    WHILE TRUE DO ! Robot does nothing, waits

        WaitUntil (numWPW <> 0);
        TPWrite "Python wants to do task WPW = "\Num:=numWPW;
        numWRD := numWPW;
        TPWrite "RAPID DOES task WRD = "\Num:=numWRD;
        numWPW := 0;

        TEST numWRD
        CASE -1:
            TPWrite "Exiting MainLoop";
            RETURN;

!            CASE 0:
!                TPWrite "CASE 0";
!                WaitTime 0.1;

        CASE 1:
            TPWrite "Moving to camera position";
            MoveToCameraPosition;

        CASE 2:
            TPWrite "Picking puck at (" + NumToStr(numPuckX, 1) + ","
               + NumToStr(numPuckY, 1) + ")";
            PickPuck numPuckX, numPuckY;

        CASE 3:
            TPWrite "Placing puck at (" + NumToStr(numPlaceX, 1) + ","
               + NumToStr(numPlaceY, 1) + ")";
            PlacePuck numPlaceX, numPlaceY;

        CASE 4:
            TPWrite "Moving to home position";
            MoveJ Offs(p0,0,0,safeHeight), vFast, z20,
               tGripper\WObj:=wobjTableN;

        DEFAULT:
            TPWrite "Unknown task number "\Num:=numWRD;
            WaitTime 0.1;
        ENDTEST
    ENDWHILE
ENDPROC

PROC MoveToCameraPosition()
    ! Move to position for capturing image
    TPWrite "Moving to camera position FUNCTION";
    MoveJ Offs(p0,0,0,cameraHeight), vFast, fine,
       tCamera\WObj:=wobjTableN;
    WaitTime 0.5; ! Time for camera to stabilize
ENDPROC
```

```
    PROC PickPuck(num x, num y)
        ! Approach puck from safe height
        MoveJ Offs(p0,x + sideOffset, y, safeHeight), vFast, z0,
            tGripper\WObj:=wobjTableN;
        ! Go down on the side
        MoveL Offs(p0,x + sideOffset, y, puckHeight), vFast, z0,
            tGripper\WObj:=wobjTableN;
        WaitTime 0.5;
        ! Take the puck
        MoveJ Offs(p0, x, y, puckHeight), v10, z0,
            tGripper\WObj:=wobjTableN;
        WaitTime 1;
        closeGripper(TRUE);

        ! Move down to puck
        !MoveL Offs(p0,x,y,-230), vSlow, z10, tGripper\WObj:=wobjTableN;

        ! Close gripper
        !CloseGripper(TRUE);
        WaitTime 0.5;

        ! Lift puck
        MoveL Offs(p0,x,y,safeHeight), vFast, z0,
            tGripper\WObj:=wobjTableN;
        numWRD := 0;
    ENDPROC


    PROC PlacePuck(num x, num y)
        ! Approach place position from safe height
        !aux.rot := [qxPlace, qyPlace, qzPlace, qwPlace];
        MoveJ Offs(aux,x,y,0), vFast, z10, tGripper\WObj:=wobjTableN;

        MoveJ RelTool(aux, 0, 0, 0, \Rz:=-anglePlace), vFast, z10,
            tGripper\WObj:=wobjTableN;

        ! Move down to place position
        MoveL Offs(pPlace1,x,y, puckHeight+heightPlace), v50, fine,
            tGripper\WObj:=wobjTableN;

        WaitTime 1;
        ! Open gripper
        CloseGripper(FALSE);
        WaitTime 0.5;

        ! Lift up
        MoveL Offs(aux,x,y,safeHeight), vFast, fine,
            tGripper\WObj:=wobjTableN;
        numWRD := 0;
    ENDPROC
ENDMODULE
```

## b) Modifications

The `TEST` cases have been revised, so they fit better to our purposes. Furthermore, a `MoveToCameraPosition()` was added, which makes capturing an image more streamlined.

Finally, `MovePuck()` is now separated into `PickPuck()` and `PlacePuck()` and we added a new function to rotate the pucks whenever they are picked to be stacked in the same direction.

As for the communication protocol, a list of global variables has been introduced. These ensure secure communication and the execution of commands from the Python script.

Implementing the rotation fully was unsuccessful; however, we managed to get the correct orientation of the puck. One hurdle was that sometimes, as the gripper pushed the puck to avoid hard collisions, it rotated as well, and the other was when placing it down in the correct configuration.

# 2 Python Code

## a) Communication

To be able to transfer data from the actual robot and the Python code to manage the pictures, we created a communication channel between both, so we could process data properly. The code we used to do so is the one shown next:

```
import cv2
import numpy as np
from pyzbar.pyzbar import decode
from rwsuis import RWS

class ABBRobotController:
    def __init__(self, ip="http://152.94.160.198"):
        self.robot = RWS.RWS(ip)
        self.camera_height = 250  # mm
        self.safe_height = 300    # mm
        self.puck_height = 30     # mm
        self.count = 0            # num

    def get_robot_status(self):
        """Get current robot status"""
        return {
            "WRD": self.robot.get_rapid_variable("numWRD"),
            "WPW": self.robot.get_rapid_variable("numWPW")
        }

    def wait_for_robot(self, timeout=500):
        """Wait until robot is ready (WRD=0)"""
        import time
        start_time = time.time()
        while time.time() - start_time < timeout:
            status = self.get_robot_status()
            if int(status["WRD"]) == 0:
                return True
            time.sleep(1)
        return False

    def send_task_command(self, task_number):
        """Send task command to robot"""
        #sleep(5)
        if self.wait_for_robot():
```

```
            print("NUMWPW VALUE:", task_number)
            self.robot.request_rmmp()
            self.robot.set_rapid_variable("numWPW", task_number)
            print("RETURNING TRUE")
            return True
        return False
```

## b) Image Processing

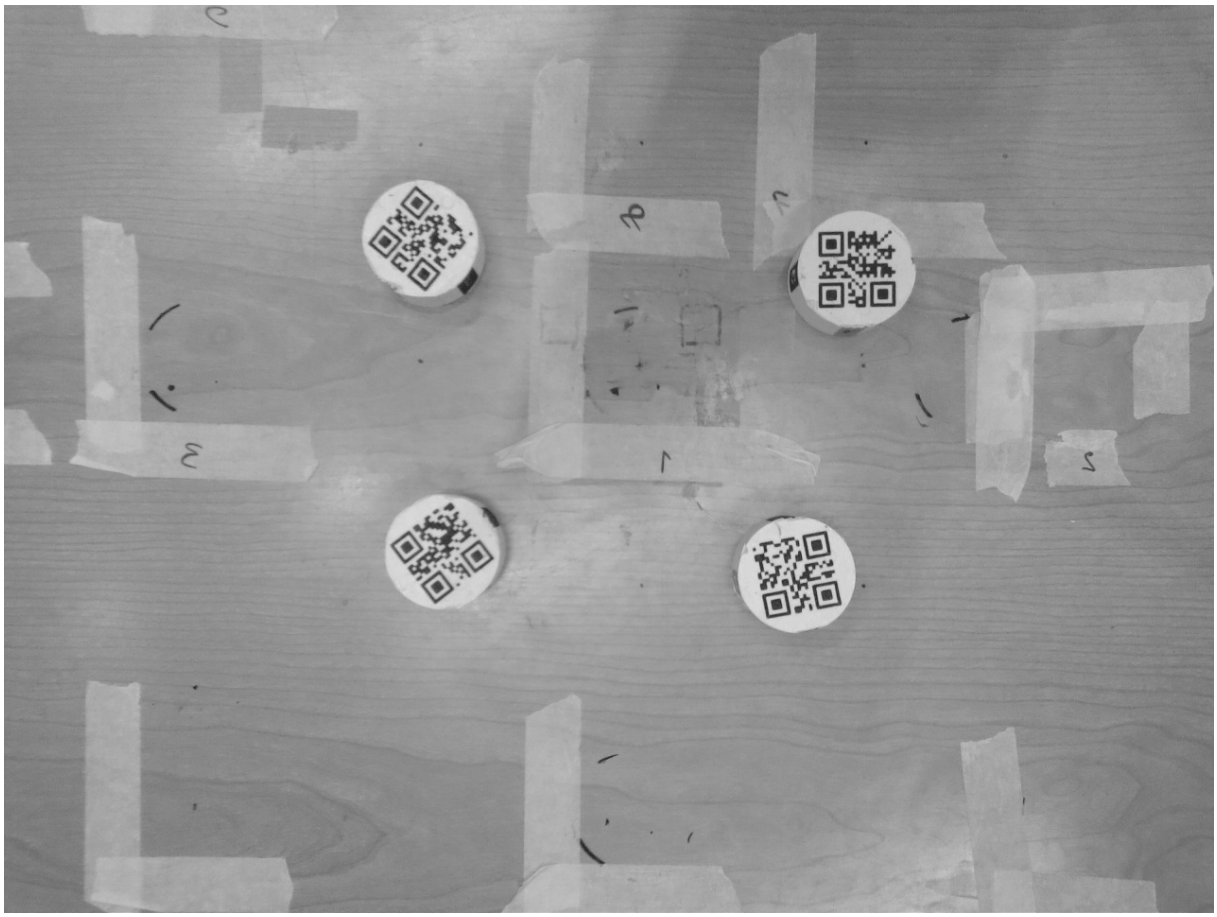

Figure 1: Captured Image from Norbert

After getting the raw footage, (e.g. Figure 1), the location and the orientation of the pucks need identification. Because each of them has a QR-code, the required data is derived from analyzing those. With the help of `pyzbar`'s `decode()` function, the corner points of every QR-code can be extracted. Manipulating these coordinates will give us the center point and orientation of every puck.

Finally, to give the actual commands to the robot, the image coordinates need to be transformed into real-life ones. With the help of scaling, offsetting and coordinate swapping, we got the needed values.

## c) Execution

When we acquire the real-life pose of every puck, the last task is to instruct Norbert to pick and place them. The procedures for these are already done in the `RAPID` code

included on 1; we just have to convey the parameters with the help of the communication protocol.

# Appendix

```
import cv2
import numpy as np
from pyzbar.pyzbar import decode
from rwsuis import RWS
import time
import math

class ABBRobotController:
    def __init__(self, ip="http://152.94.160.198"):
        self.robot = RWS.RWS(ip)
        self.camera_height = 250  # mm
        self.safe_height = 300    # mm
        self.puck_height = 30     # mm
        self.count = 0            # num

    def get_robot_status(self):
        """Get current robot status"""
        return {
            "WRD": self.robot.get_rapid_variable("numWRD"),
            "WPW": self.robot.get_rapid_variable("numWPW")
        }

    def wait_for_robot(self, timeout=500):
        """Wait until robot is ready (WRD=0)"""
        start_time = time.time()
        while time.time() - start_time < timeout:
            status = self.get_robot_status()
            if int(status["WRD"]) == 0:
                return True
            time.sleep(1)
        return False

    def send_task_command(self, task_number):
        """Send task command to robot"""
        if self.wait_for_robot():
            self.robot.request_rmmp()
            self.robot.set_rapid_variable("numWPW", task_number)
            return True
        return False

    def set_puck_position(self, x, y):
        """Set puck position for picking"""
        self.robot.set_rapid_variable("numPuckX", y)
        self.robot.set_rapid_variable("numPuckY", -x)


    def set_place_position(self, x, y, angle):
        """Set place position for puck"""
        self.robot.set_rapid_variable("numPlaceX", y)
        self.robot.set_rapid_variable("numPlaceY", -x)
        self.robot.set_rapid_variable("anglePlace", angle)
        self.robot.set_rapid_variable("heightPlace", self.count*30)

    def move_to_camera_position(self):
        """Command robot to move to camera position"""
```

```
        return self.send_task_command(1)

    def pick_puck(self, x, y):
        """Command robot to pick puck at (x,y)"""
        self.set_puck_position(x, y)
        return self.send_task_command(2)

    #def place_puck(self, x, y, qx, qy, qz, qw):
    def place_puck(self, x, y, angle):
        """Command robot to place puck at (x,y)"""
        self.set_place_position(x, y, angle)
        self.count += 1
        return self.send_task_command(3)

    def go_home(self):
        """Command robot to return to home position"""
        return self.send_task_command(4)

class PuckDetector:
    def __init__(self, camera_index=1):
        self.cap = cv2.VideoCapture(camera_index)
        self.cap.set(3, 1280)  # Width
        self.cap.set(4, 960)   # Height
        self.calibration_points = []  # For coordinate transformation

    def capture_image(self):
        """Capture image from camera"""
        sleep(5)
        ret, frame = self.cap.read()
        if ret:
            return frame
        return None

    def preprocess_image(self, image):
        """Preprocess image for better QR detection"""
        gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
        contrast = cv2.addWeighted(gray, 1, np.zeros(gray.shape,
           gray.dtype), 0, 40)
        cv2.imshow("CONTRASTED IMG", contrast)
        cv2.waitKey(0)
        return contrast

    def detect_pucks(self, image):
        """Detect pucks with QR codes in image"""
        processed = self.preprocess_image(image)
        decoded_objects = decode(processed)

        pucks = []
        for obj in decoded_objects:
            points = obj.polygon
            if len(points) == 4:
                # Calculate center and orientation
                center_x = sum([p.x for p in points]) / 4
                center_y = sum([p.y for p in points]) / 4

                # Calculate orientation from QR code corners
                dx = points[1].x - points[0].x
                dy = points[1].y - points[0].y
```

```
                angle = np.arctan2(dy, dx) * 180 / np.pi

                pucks.append({
                    "id": obj.data.decode("utf-8"),
                    "center": (center_x, center_y),
                    "corners": [(p.x, p.y) for p in points],
                    "angle": angle
                })

        return pucks

    def calibrate_camera(self, image_points, table_points):
        """Calculate transformation matrix from image to table
           coordinates"""
        # Add homogeneous coordinate (1)
        img_pts = np.array([(x, y, 1) for x, y in image_points])
        tbl_pts = np.array([(x, y, 1) for x, y in table_points])

        # Calculate transformation matrix
        T, _, _, _ = np.linalg.lstsq(img_pts, tbl_pts, rcond=None)
        return T  # Transpose to get correct matrix

    def image_to_table(self, image_point, T_matrix):
        """Convert image coordinates to table coordinates"""
        m = 32/77.7946
        img_pt = np.array([image_point[0], image_point[1], 1])
        tbl_pt = np.dot(T_matrix, img_pt)
        return ((tbl_pt[0]-m*640)-5), ((tbl_pt[1]-m*480)+45)

def main():
    # Initialize components
    robot = ABBRobotController()
    detector = PuckDetector()
    count = 0

    # Example calibration (replace with actual calibration points)
    # These are known points on table and their corresponding image
    m = 32/77.7946
    T_matrix = [[m, 0, 0],
                [0, m, 0],
                [0, 0, 1]]
    # Calculate transformation matrix

    print("Transformation matrix:")
    print(T_matrix)

    # Main loop
    try:
        while True:
            input("Press Enter to capture image and detect pucks...")

            # Move robot to camera position
            if not robot.move_to_camera_position():
                print("Failed to move robot to camera position")
                continue

            # Capture image
            image = detector.capture_image()
```

```
            if image is None:
                print("Failed to capture image")
                continue

            # Detect pucks
            pucks = detector.detect_pucks(image)
            print(f"Detected {len(pucks)} pucks")

            for puck in pucks:
                # Convert to table coordinates
                table_x, table_y = detector.image_to_table(puck["center"],
                    T_matrix)
                print(f"Puck {puck['id']} at table position:
                    ({table_y:.1f}, {-table_x:.1f})")
                print("Puck corners", puck['corners'])
                print("Puck angle", puck['angle'])
                angle = puck['angle']
                angle = float(angle)

                # Pick and place puck
                if robot.pick_puck(table_x, table_y):
                    # Place in default position (adjust as needed)
                    print("PUCK BEING PICKED")
                    place_x = 0
                    place_y = 0
                    print("PLACE X AND Y", place_x, place_y)
                    robot.place_puck(place_x, place_y, angle)
                    print("Successfully placed puck")
                else:
                    print("Failed to pick puck")


    except KeyboardInterrupt:
        print("Program stopped by user")

    # Release camera
    detector.cap.release()

if __name__ == "__main__":
    main()
```